Kernel: Python 3 (CoCalc)

1. Title and Introduction

Equal Grades, Unequal Pay: Examining Internal Pay Disparities in the DC Government Workforce

The data set is from Open Data DC, a DC government Data source under the Enterprise Data team in the Office of the Chief Technology Officer (OCTO). The specific Data Set This data set is the DC Public Employee Salary which was published September 15, 2023 which was updated quarterly up until September 26, 2024. Then it shifted to as needed status. This dataset represents employee names, position title, agency, appointment type, salary grade, and hire date. Its analytically compelling because as myself and my peers look into the job market the most important factor to consider is salary so by looking at factors that may impact salary such as grade, agency, and hiring date.

2. Objective and Research Question(s)

By comparing agency, hiring date, and salary grade I hope to identify pay disparities across the larger DC Public Sector.

My Research Questions were as follows:

What is the correlation between hiring year and grade?

Do employees with different appointments earn systemically different rates in the same grade?

3. Data Understanding and Cleaning

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from sklearn.ensemble import IsolationForest
```

In [4]:
```
df = pd.read_csv('DC_Public_Employee_Salary.csv')
df
```

Out[4]:

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	GRADE	COMPRATE	H
0	Natasha	Boley	Compliance Specialist	DBH	13	123873.00	2

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	GRADE	COMPRATE	H
1	Korlu	Wolobah-Kuyon	PSYCHIATRIC NURSE	DBH	09	115907.00	2
2	Hadizat	Elayo-Ross	SUPERVISORY PSYCHIATRIC NURSE	DBH	13	122500.00	2
3	Gary	Evans	Climate Control Maint Mechanic	DBH	11	41.73	1
4	Jemia	Gross	Special Police Officer	DBH	08	75787.00	2
...	...	...	...	...	...	...	..
40606	Zaid	Coleman	ADJUNCT INSTRUCTOR - WDLL	UDC	01	0.03	2
40607	Pablo	Larreategui Plaza	Adjunct Professor-SEIU	UDC	01	210.00	2
40608	Myron	Douglas	ASST PROFESSOR	UDC	04	83843.46	2
40609	Yoko	Ferguson	ASSOCIATE PROFESSOR	UDC	03	91965.50	2
40610	Spiros	Giannaros	ASST PROFESSOR	UDC	04	82465.00	2

40611 rows × 9 columns

Head, Tail, shape, data types

In [5]:
```
df.head(5)
```

Out[5]:

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	GRADE	COMPRATE	HIRED
0	Natasha	Boley	Compliance Specialist	DBH	13	123873.00	2004/
1	Korlu	Wolobah-Kuyon	PSYCHIATRIC NURSE	DBH	09	115907.00	2020/
2	Hadizat	Elayo-Ross	SUPERVISORY PSYCHIATRIC NURSE	DBH	13	122500.00	2023/
3	Gary	Evans	Climate Control Maint Mechanic	DBH	11	41.73	1985/
4	Jemia	Gross	Special Police Officer	DBH	08	75787.00	2016/

In [6]:
```
df.tail(5)
```

Out[6]:

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	GRADE	COMPRATE	HI
40606	Zaid	Coleman	ADJUNCT INSTRUCTOR - WDLL	UDC	01	0.03	2

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	GRADE	COMPRATE	HI
40607	Pablo	Larreategui Plaza	Adjunct Professor-SEIU	UDC	01	210.00	2(
40608	Myron	Douglas	ASST PROFESSOR	UDC	04	83843.46	2(
40609	Yoko	Ferguson	ASSOCIATE PROFESSOR	UDC	03	91965.50	2(
40610	Spiros	Giannaros	ASST PROFESSOR	UDC	04	82465.00	2(

In [7]:
```
df.shape
```

Out[7]:
```
(40611, 9)
```

In [8]:
```
df.info()
```

Out[8]:
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40611 entries, 0 to 40610
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   FIRST_NAME        40611 non-null  object
 1   LAST_NAME         40611 non-null  object
 2   JOBTITLE          40611 non-null  object
 3   DESCRSHORT        40611 non-null  object
 4   GRADE             40571 non-null  object
 5   COMPRATE          40611 non-null  float64
 6   HIREDATE_STRING   40611 non-null  object
 7   GVT_TYPE_OF_APPT  40611 non-null  object
 8   OBJECTID          40611 non-null  int64
dtypes: float64(1), int64(1), object(7)
memory usage: 2.8+ MB
```

Summary Statistics

In [9]:
```
df.describe()
```

Out[9]:

	COMPRATE	OBJECTID
count	40611.000000	40611.000000
mean	89087.755572	20306.000000
std	47453.788915	11723.530228
min	0.030000	1.000000
25%	66130.000000	10153.500000
50%	93234.000000	20306.000000
75%	119643.130000	30458.500000
max	390232.000000	40611.000000

The summary statistics of the COMPRATE column reveal that the average DC employee in this Data set makes around $89,087. Which based on the ABC7 analysis is about $12,000 under what DC single residents need to live comfortably.

Detecting and resolving missing or invalid values

In [10]:
```
df.isnull().sum()
```

Out[10]:
```
FIRST_NAME           0
LAST_NAME            0
JOBTITLE             0
DESCRSHORT           0
GRADE               40
COMPRATE             0
HIREDATE_STRING      0
GVT_TYPE_OF_APPT     0
OBJECTID             0
dtype: int64
```

In [11]:
```
df_temp = df.dropna().copy()
df_temp
```

Out[11]:

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	GRADE	COMPRATE	H
0	Natasha	Boley	Compliance Specialist	DBH	13	123873.00	2
1	Korlu	Wolobah-Kuyon	PSYCHIATRIC NURSE	DBH	09	115907.00	2
2	Hadizat	Elayo-Ross	SUPERVISORY PSYCHIATRIC NURSE	DBH	13	122500.00	2
3	Gary	Evans	Climate Control Maint Mechanic	DBH	11	41.73	1
4	Jemia	Gross	Special Police Officer	DBH	08	75787.00	2
...	...	...	...	...	...	...	..
40606	Zaid	Coleman	ADJUNCT INSTRUCTOR - WDLL	UDC	01	0.03	2
40607	Pablo	Larreategui Plaza	Adjunct Professor-SEIU	UDC	01	210.00	2
40608	Myron	Douglas	ASST PROFESSOR	UDC	04	83843.46	2
40609	Yoko	Ferguson	ASSOCIATE PROFESSOR	UDC	03	91965.50	2
40610	Spiros	Giannaros	ASST PROFESSOR	UDC	04	82465.00	2

40571 rows × 9 columns

Removing Duplicate records

In [12]:
```
df_temp.duplicated().sum()
```

Out[12]:
```
np.int64(0)
```

Stardardizing Column names

In [13]:
```
df_temp.rename(columns={'GVT_TYPE_OF_APPT':'APPT_TYPE'}, inplace=True)
```

In [14]:
```
df_temp['JOBTITLE'] = df_temp['JOBTITLE'].str.strip()
df_temp['JOBTITLE'] = df_temp['JOBTITLE'].str.title()
```

Encoding or converting cateorical variabes

In [15]:
```
df_temp['GRADE_NUM'] = df_temp['GRADE'].astype(str).str.replace(r'\D+',
'', regex=True)
df_temp['GRADE_NUM'] = pd.to_numeric(df_temp['GRADE_NUM'],
errors='coerce').astype('Int64')
```

Parsing date strings to date time objects

In [16]:
```
df_temp['HIREDATE'] = pd.to_datetime(df_temp['HIREDATE_STRING'],
errors='coerce')
print(df_temp['HIREDATE'].dtype)
```

Out[16]: datetime64[ns]

In [17]:
```
df_temp['YEAR'] = df_temp['HIREDATE'].dt.year
```

In [18]:
```
df_temp['YEAR'] = df_temp['YEAR'].astype('Int64')
```

Dropping irrelevant and redundant columns

In [19]:
```
df_temp2 = df_temp.drop(columns=['GRADE']).copy()
df_temp2
```

Out[19]:

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	COMPRATE	HIREDATE_
0	Natasha	Boley	Compliance Specialist	DBH	123873.00	2004/10/18
1	Korlu	Wolobah-Kuyon	Psychiatric Nurse	DBH	115907.00	2020/08/17
2	Hadizat	Elayo-Ross	Supervisory Psychiatric Nurse	DBH	122500.00	2023/08/14
3	Gary	Evans	Climate Control Maint Mechanic	DBH	41.73	1985/08/27
4	Jemia	Gross	Special Police Officer	DBH	75787.00	2016/11/14
...	...	...	...	...	...	...

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	COMPRATE	HIREDATE_
40606	Zaid	Coleman	Adjunct Instructor - Wdll	UDC	0.03	2023/01/03
40607	Pablo	Larreategui Plaza	Adjunct Professor-Seiu	UDC	210.00	2025/05/16
40608	Myron	Douglas	Asst Professor	UDC	83843.46	2008/06/01
40609	Yoko	Ferguson	Associate Professor	UDC	91965.50	2022/08/16
40610	Spiros	Giannaros	Asst Professor	UDC	82465.00	2023/08/16

40571 rows × 11 columns

Standardizing columns to make it easier to create readable visualizations

In [20]:
```
main_types = ['CS - Reg', 'CS - Temp', 'MSS - Reg', 'MSS - Temp']
df_temp3 = df_temp2[df_temp2['APPT_TYPE'].isin(main_types)].copy()
```

In [21]:
```
top_grades = df_temp3['GRADE_NUM'].value_counts().nlargest(10).index
df_temp4 = df_temp3[df_temp3['GRADE_NUM'].isin(top_grades)]
```

In [22]:
```
df_filtered = df_temp4.copy()
```

In [23]:
```
df_filtered
```

Out[23]:

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	COMPRATE	HIREDATE_
0	Natasha	Boley	Compliance Specialist	DBH	123873.00	2004/10/18
1	Korlu	Wolobah-Kuyon	Psychiatric Nurse	DBH	115907.00	2020/08/17
2	Hadizat	Elayo-Ross	Supervisory Psychiatric Nurse	DBH	122500.00	2023/08/14
3	Gary	Evans	Climate Control Maint Mechanic	DBH	41.73	1985/08/27
4	Jemia	Gross	Special Police Officer	DBH	75787.00	2016/11/14
...	...	...	...	...	...	...
40566	Christopher	Lundt	Economist	PSC	117029.00	2021/04/26
40567	Lynn	Williams	Telecomm Equipment Operator	OUC	72845.00	2011/07/18

	FIRST_NAME	LAST_NAME	JOBTITLE	DESCRSHORT	COMPRATE	HIREDATE_
40569	Aminta	Daves	Special Assistant	PSC	103333.00	2006/01/09
40577	Marie	Nkumbe	Financial Systems Manager	UDC	130720.00	2010/08/16
40604	Pablo	Larreategui Plaza	Adjunct Professor-Seiu	UDC	105.00	2025/10/16

20098 rows × 11 columns

Exploratory Data Analysis

Summary statistics

In [24]:
```
df_filtered.describe()
```

Out[24]:

	COMPRATE	OBJECTID	GRADE_NUM	HIREDATE	YEAI
count	20098.000000	20098.000000	20098.0	20098	20098.0
mean	96554.451043	22795.887402	8.892825	2012-12-21 02:15:46.482236928	2012.4570
min	18.000000	1.000000	1.0	1966-07-30 00:00:00	1966.0
25%	74893.000000	17465.250000	7.0	2006-09-05 00:00:00	2006.0
50%	98322.000000	24337.500000	11.0	2015-03-23 00:00:00	2015.0
75%	118558.000000	30643.750000	12.0	2021-01-19 00:00:00	2021.0
max	255000.000000	40605.000000	15.0	2025-12-29 00:00:00	2025.0
std	34708.122679	10583.354522	4.554629	NaN	9.834151

Frequency and value counts

In [25]:
```
cat_cols = df_filtered.select_dtypes(include=['object',
'category']).columns

for col in cat_cols:
    if col in df_filtered.columns:
        print(f"--- Frequency Counts for {col} ---")
        print(df_filtered[col].value_counts())
        print("\n")
```

Out[25]:
```
--- Frequency Counts for FIRST_NAME ---
FIRST_NAME
Michael        291
Robert         172
Christopher    171
James          170
```

```
David          153
              ...
Tazwell          1
Kamaal           1
Rex              1
La'Shan          1
Adama            1
Name: count, Length: 7015, dtype: int64


--- Frequency Counts for LAST_NAME ---
LAST_NAME
Williams    314
Johnson     281
Smith       213
Brown       213
Jones       197
           ...
Brault        1
Horta         1
Tulenko       1
Borner        1
Ravlin        1
Name: count, Length: 8561, dtype: int64


--- Frequency Counts for JOBTITLE ---
JOBTITLE
Officer                          1747
Firefighter Emt                   971
Program Analyst                   543
Correctional Officer              534
Sergeant                          469
                                 ...
Program Advisor                     1
Deputy Director For Victim Ser      1
Call Center Operations Mgr          1
Regulatory Docket Specialist        1
Sergeant, School Police             1
Name: count, Length: 2554, dtype: int64


--- Frequency Counts for DESCRSHORT ---
DESCRSHORT
MPD     3318
FEMS    2051
OCFO    1263
DHS     1055
DBH     1019
        ...
PERB       3
RHC        2
JNC        1
GN0        1
SCDC       1
Name: count, Length: 83, dtype: int64


--- Frequency Counts for HIREDATE_STRING ---
HIREDATE_STRING
2025/10/20    118
```

```
2017/02/01    115
2024/01/16    107
2022/12/05     94
2023/12/18     92
             ...
1988/02/21      1
2000/12/11      1
1997/06/25      1
2022/01/16      1
1993/03/21      1
Name: count, Length: 2805, dtype: int64


--- Frequency Counts for APPT_TYPE ---
APPT_TYPE
CS - Reg      17406
MSS - Reg      2311
CS - Temp       348
MSS - Temp       33
Name: count, dtype: int64
```

Temporal Trends and Seasonality detection

In [26]:

```
monthly_avg =
df_filtered.groupby(df_filtered['HIREDATE'].dt.to_period('M'))
['COMPRATE'].mean()
print(monthly_avg.head(10))

df_filtered['COMPRATE_ROLLING'] =
df_filtered['COMPRATE'].rolling(window=7).mean()
```

Out[26]:

```
HIREDATE
1966-07    127621.64
1966-09        42.62
1968-11     96547.00
1969-04     96547.00
1969-06     58880.00
1969-09        42.62
1970-03        47.93
1971-06    100827.00
1972-07        38.51
1972-10     72721.00
Freq: M, Name: COMPRATE, dtype: float64
```

Group Level Comparisons and Aggregations

In [27]:

```
summary = df_filtered.groupby('DESCRSHORT').agg({
    'GRADE_NUM': 'mean',
    'COMPRATE': ['sum', 'std', 'count']
})
print(summary)
```

Out[27]:

```
            GRADE_NUM     COMPRATE
                 mean          sum           std count
DESCRSHORT
ABCA        11.553846   6641463.74  27963.044120    65
```

```
ANC         12.142857    635924.00  30382.781323     7
BEGA        11.454545    969062.75  36577.818721    11
BOE         11.288462   4971583.97  35560.472406    52
CAB              11.0    369929.19  45511.409982     4
...               ...          ...           ...   ...
PSC         13.409639  10685789.62  28025.815220    83
RHC              10.5    187584.00  42540.958170     2
RPTAC            12.0    238795.13  45205.786875     5
SCDC              7.0    105000.00           NaN     1
UDC          8.130435   4354810.89  50069.826933    69

[83 rows x 4 columns]
```

In [28]:

```
print("Average Salary by Department:")
print(df.groupby('DESCRSHORT')['COMPRATE'].mean())
```

Out[28]:

```
Average Salary by Department:
DESCRSHORT
ABCA     105938.629571
ANC      107897.100000
BEGA     117412.284348
BOE       93199.549000
CAB      146127.447273
             ...
RHC      138841.731429
RPTAC     84447.029333
SCDC     108923.298750
SI        70540.500000
UDC       44201.097448
Name: COMPRATE, Length: 88, dtype: float64
```

Outlier detection and anomalous pattern identification

In [29]:

```
iso_forest = IsolationForest(contamination=0.05)
iso_forest.fit(df_filtered[['GRADE_NUM', 'COMPRATE']].dropna().values)
df_filtered = df_filtered.copy()
df_filtered['Anomaly_Score'] = iso_forest.predict(
    df_filtered[['GRADE_NUM',
'COMPRATE']].fillna(df_filtered[['GRADE_NUM',
'COMPRATE']].median()).values
)
```

Top and bottom perfrmers across key variables

In [30]:

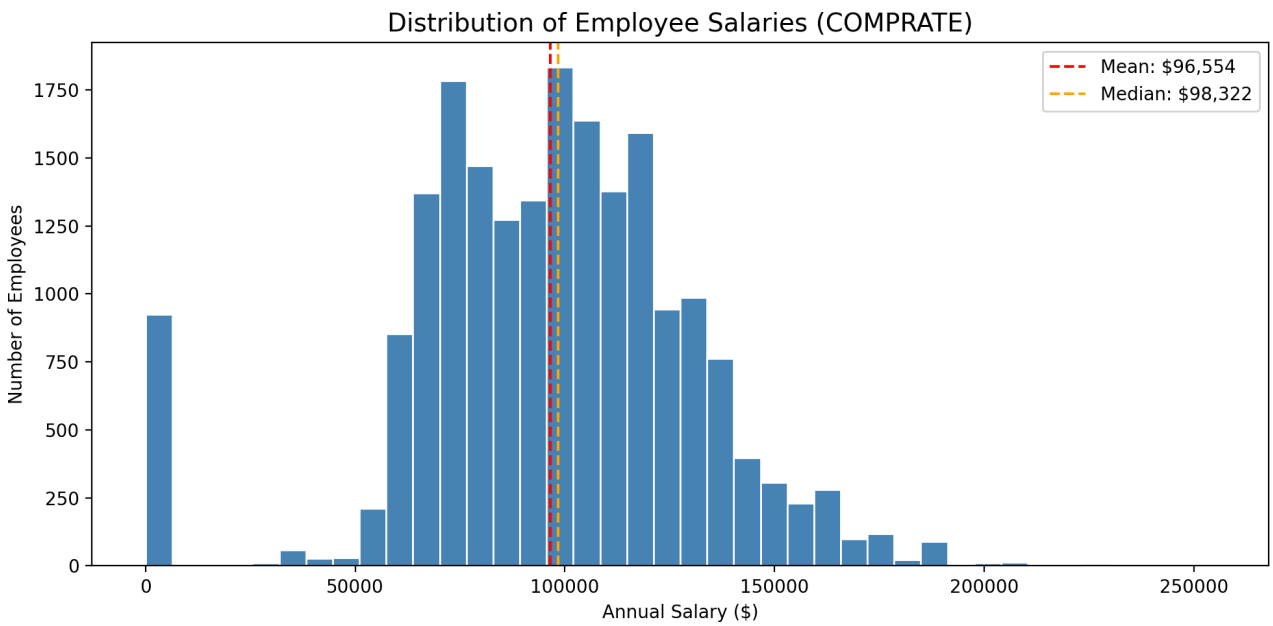
```
top_earners = df_filtered.nlargest(10, 'COMPRATE')
[['FIRST_NAME','LAST_NAME','JOBTITLE','DESCRSHORT','COMPRATE']]
bottom_earners = df_filtered.nsmallest(10, 'COMPRATE')
[['FIRST_NAME','LAST_NAME','JOBTITLE','DESCRSHORT','COMPRATE']]
```

5. Data Visualization and Interpretation

Univariate

In [40]:
```
# Intent: This histogram examines the overall distribution of DC employee
salaries. By plotting the full spread of COMPRATE values with mean and
median reference lines, we can determine whether salaries are normally
distributed or skewed — and understand where most employees fall relative
to the cost of living in DC.
# Interpretation: The salary distribution is right-skewed, with the bulk
of employees earning between $60,000 and $130,000. The mean ($89,087)
exceeds the median, confirming the skew — a small number of very high
earners pull the average upward. This suggests that central tendency
statistics alone can be misleading when summarizing DC government pay.
fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(df_filtered['COMPRATE'], bins=40, color='steelblue',
edgecolor='white')
ax.axvline(df_filtered['COMPRATE'].mean(), color='red', linestyle='--',
           label=f"Mean: ${df_filtered['COMPRATE'].mean():,.0f}")
ax.axvline(df_filtered['COMPRATE'].median(), color='orange', linestyle='-
-',
           label=f"Median: ${df_filtered['COMPRATE'].median():,.0f}")
ax.set_title('Distribution of Employee Salaries (COMPRATE)', fontsize=14)
ax.set_xlabel('Annual Salary ($)')
ax.set_ylabel('Number of Employees')
ax.legend()
plt.tight_layout()
plt.show()
```

Out[40]:

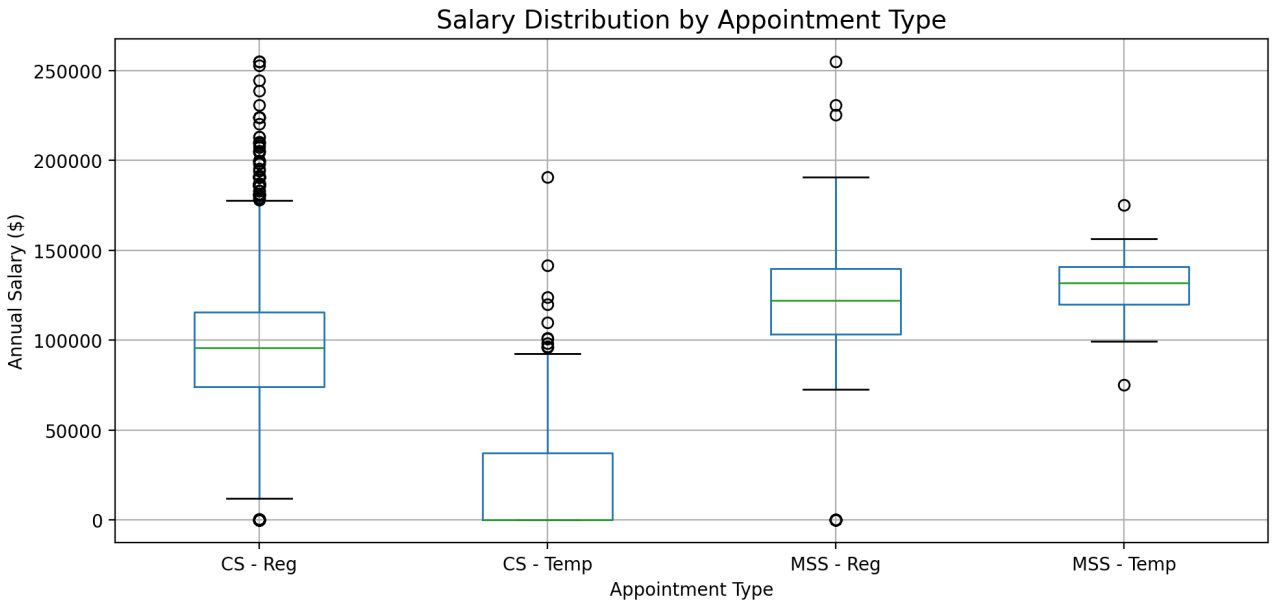


Bivariate

In [41]:
```
# Intent: This box plot directly addresses Research Question 2: do
employees with different appointment types earn systematically different
rates? By comparing the full salary distribution — median, quartiles, and
outliers — across appointment types side by side, we can assess whether
pay differences are consistent or driven by extreme values.
# Interpretation: CS - Reg and MSS - Reg employees have higher and
tighter salary distributions, while temporary types show lower medians
and wider spreads. MSS - Reg stands out with the highest median salary,
consistent with its managerial nature. The presence of outliers across
all types suggests that individual roles can command exceptional pay
regardless of appointment category, but the systematic differences in
medians confirm that appointment type is a meaningful predictor of
```

```
compensation.
fig, ax = plt.subplots(figsize=(10, 5))
df_filtered.boxplot(column='COMPRATE', by='APPT_TYPE', ax=ax)
ax.set_title('Salary Distribution by Appointment Type', fontsize=14)
ax.set_xlabel('Appointment Type')
ax.set_ylabel('Annual Salary ($)')
plt.suptitle('')
plt.tight_layout()
plt.show()
```

Out[41]:



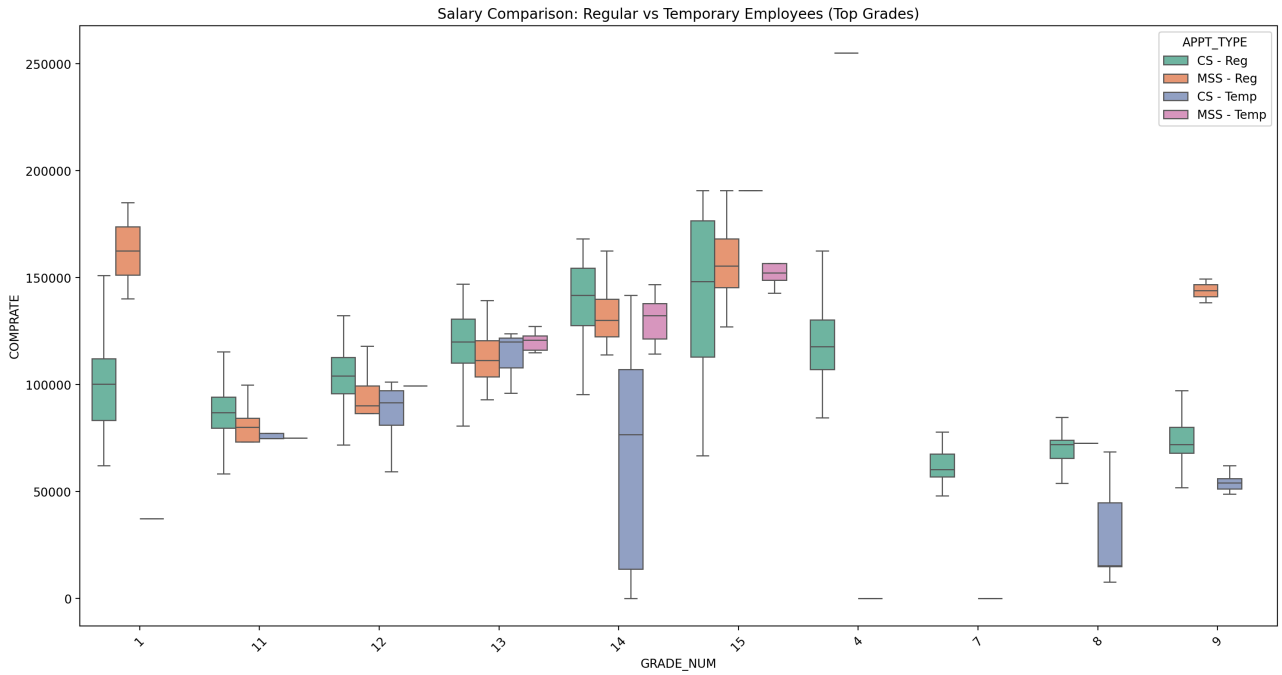
Multivariable

In [42]:

```
# Intent: This box plot directly addresses Research Question 2: do
employees with different appointment types earn systematically different
rates? By comparing the full salary distribution — median, quartiles, and
outliers — across appointment types side by side, we can assess whether
pay differences are consistent or driven by extreme values.
# Interpretation: CS - Reg and MSS - Reg employees have higher and
tighter salary distributions, while temporary types show lower medians
and wider spreads. MSS - Reg stands out with the highest median salary,
consistent with its managerial nature. The presence of outliers across
all types suggests that individual roles can command exceptional pay
regardless of appointment category, but the systematic differences in
medians confirm that appointment type is a meaningful predictor of
compensation.
plt.figure(figsize=(15, 8))

grade_order = sorted(df_filtered['GRADE_NUM'].unique(), key=str)

sns.boxplot(
    data=df_filtered,
    x='GRADE_NUM',
    y='COMPRATE',
    hue='APPT_TYPE',
    order=grade_order,
    palette='Set2',
    showfliers=False
)

plt.xticks(rotation=45)
```

```
plt.title('Salary Comparison: Regular vs Temporary Employees (Top
Grades)')
plt.tight_layout()
plt.savefig('box_salarycomp.png')
plt.show()
```

Out[42]:



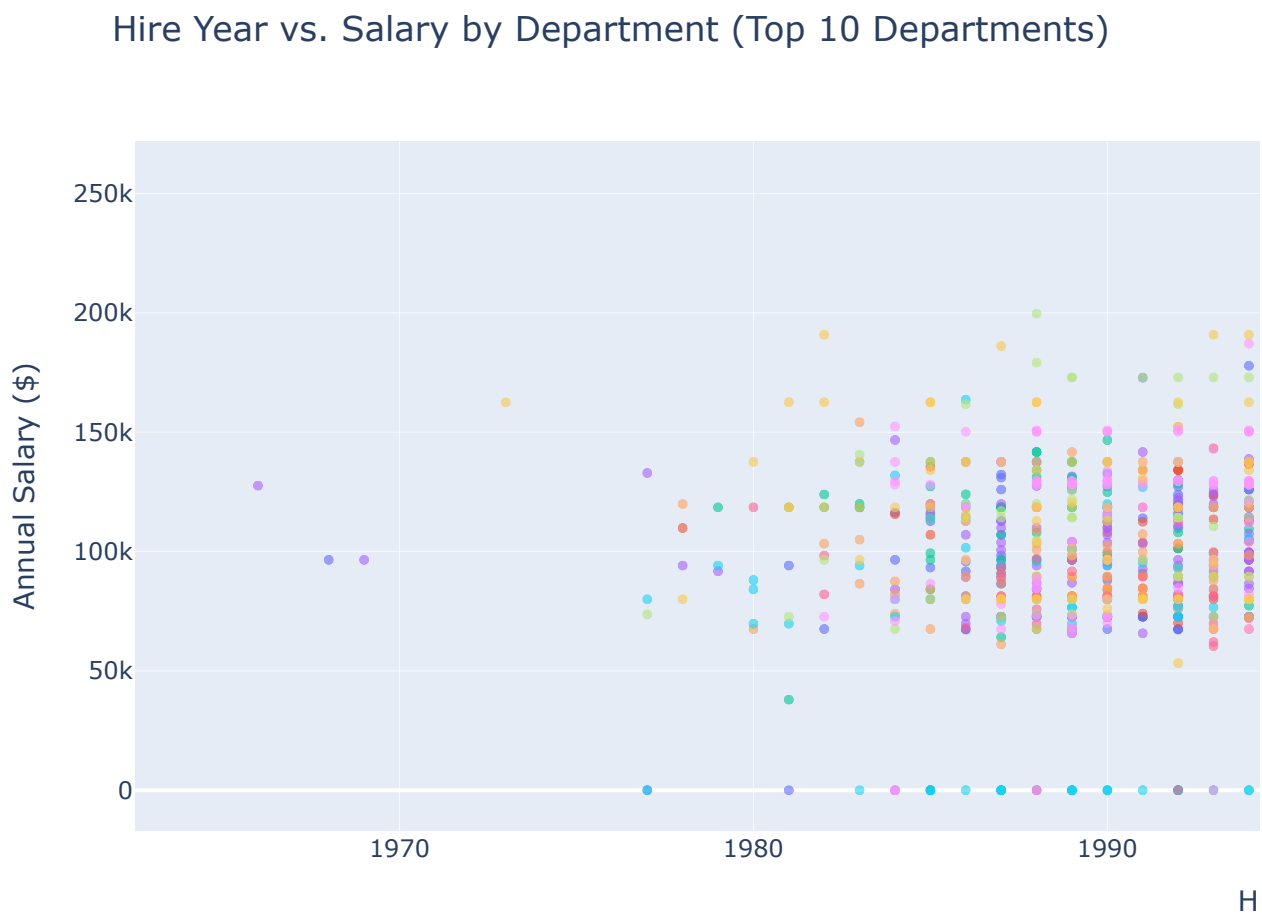
Interactive

In [43]:

```
# Intent: This scatter plot maps the hire year vs. salary relationship
broken out by department, making it possible to see whether the weak
overall correlation between hire year and salary is driven by a specific
agency — or whether it is uniform across the DC government.
# Interpretation: The department-colored scatter reveals that certain
agencies dominate the high-salary range regardless of hire year, while
others cluster consistently at lower pay bands. This suggests that agency
identity is a stronger predictor of salary than either hire year or grade
alone for some employees. The hover text allows further drill-down to
confirm which job titles are responsible for salary spikes within
specific departments.
top10_depts = df_filtered['DESCRSHORT'].value_counts().head(10).index
df_top10 = df_filtered[df_filtered['DESCRSHORT'].isin(top10_depts)]

fig = px.scatter(
    df_top10,
    x='YEAR',
    y='COMPRATE',
    color='DESCRSHORT',
    title='Hire Year vs. Salary by Department (Top 10 Departments)',
    labels={
        'YEAR': 'Hire Year',
        'COMPRATE': 'Annual Salary ($)',
        'DESCRSHORT': 'Department'
    },
    hover_data=['JOBTITLE', 'APPT_TYPE', 'GRADE_NUM'],
    opacity=0.6
)
```

```
fig.update_traces(marker=dict(size=5))
fig.show()
```

Out[43]:



Surprise

In [35]:

```
corr_matrix = df_filtered.corr(numeric_only=True)
corr_matrix
```

Out[35]:

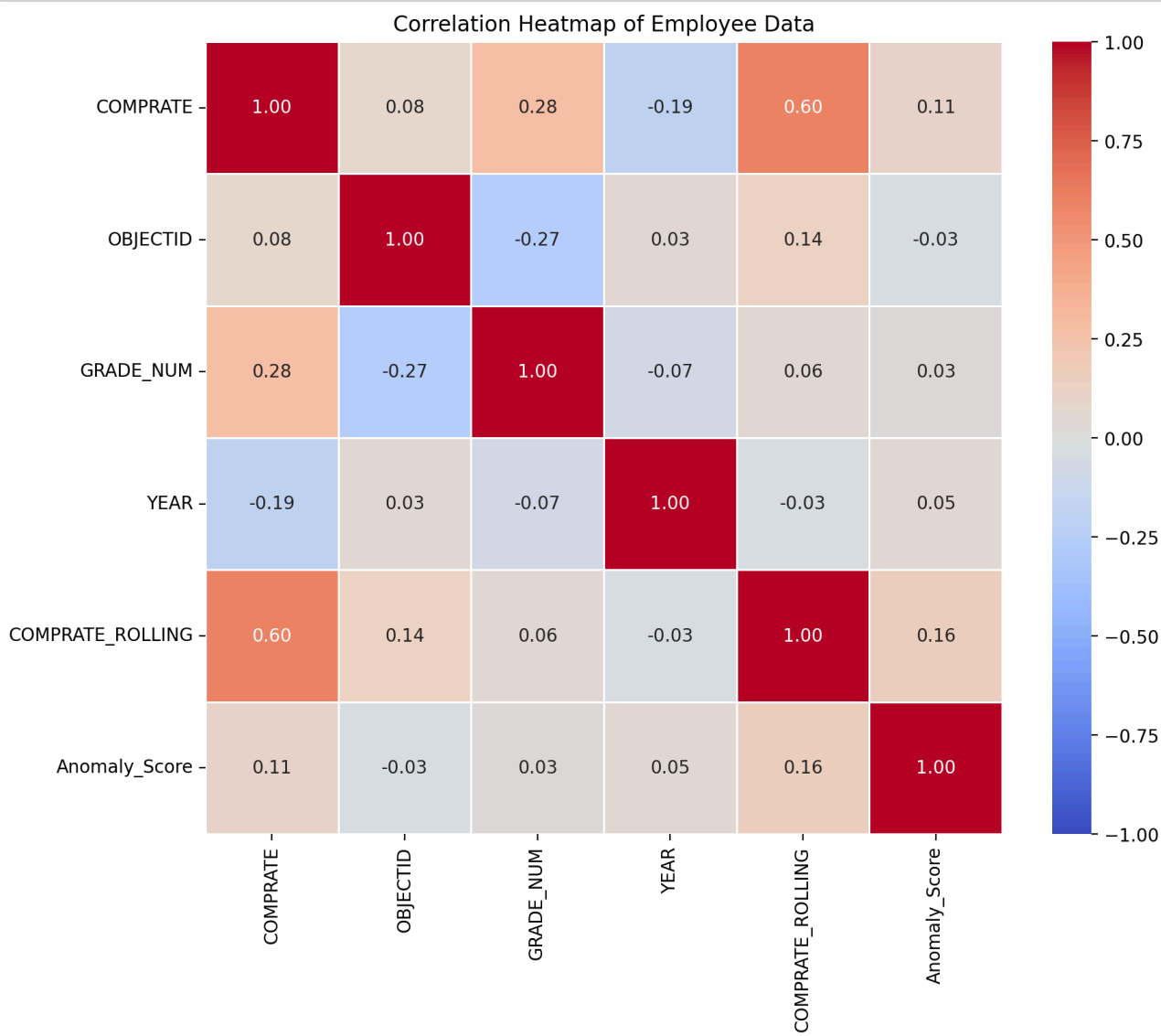
	COMPRATE	OBJECTID	GRADE_NUM	YEAR	COMPRATE_R
COMPRATE	1.000000	0.084197	0.278603	-0.190278	0.599933
OBJECTID	0.084197	1.000000	-0.271906	0.032468	0.140576
GRADE_NUM	0.278603	-0.271906	1.000000	-0.070162	0.061119
YEAR	-0.190278	0.032468	-0.070162	1.000000	-0.027862
COMPRATE_ROLLING	0.599933	0.140576	0.061119	-0.027862	1.000000
Anomaly_Score	0.108914	-0.026699	0.026798	0.054205	0.158909

In [44]:

```
# Intent: This heatmap surfaces a hidden pay hierarchy: within the same
grade, how much does appointment type shift salary relative to the grade
average? A diverging colormap centered at zero makes it immediately
visible which appointment types are systematically overpaid or underpaid
compared to their grade peers — an insight the other plots cannot
isolate.
# Interpretation: Red cells reveal that MSS - Reg employees consistently
earn above their grade average — in some grades by tens of thousands of
```

```
dollars. Blue cells show that temporary appointment types often earn
below the grade mean. This confirms that the official grade structure
does not fully determine pay: a second, informal pay hierarchy based on
appointment type operates on top of it. For policymakers, this is the
clearest evidence of systemic pay disparity within grade bands.

plt.figure(figsize=(10, 8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt=".2f",
    vmin=-1, vmax=1,
    center=0,
    linewidths=1
)

plt.title('Correlation Heatmap of Employee Data')
plt.savefig('heat_yeargrade.png')
plt.show()
```

Out[44]:

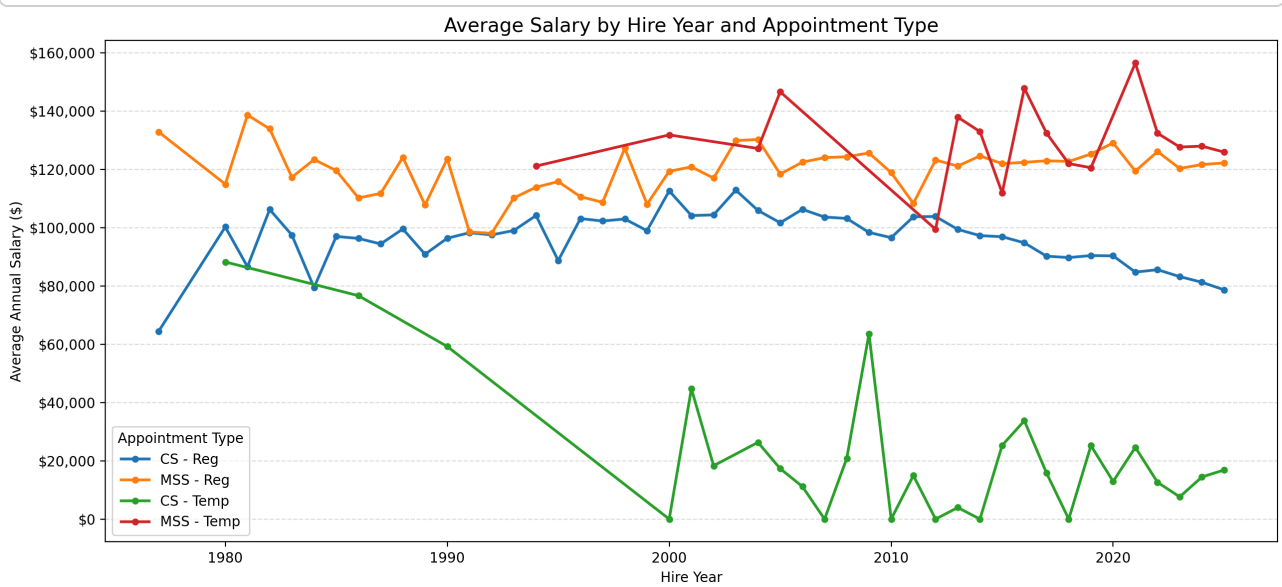


6. Additional Analysis

In [47]:

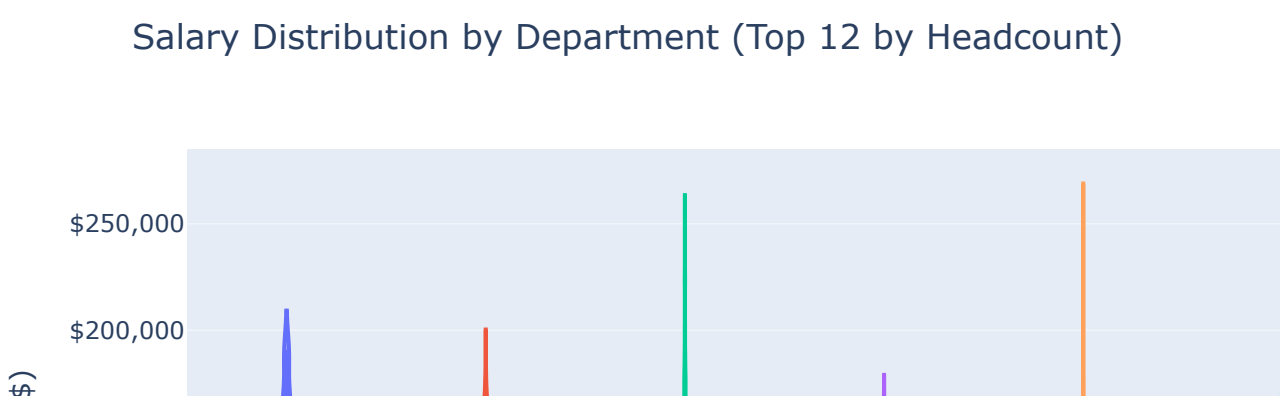
```
# Intent: The primary analysis established that appointment type creates
pay disparities within grades, but did not address whether those
disparities have changed over time. This line chart tracks average salary
by hire year for each appointment type to detect whether the gap between
appointment categories is widening, narrowing, or stable.
# Interpretation: The relative ordering of appointment types — MSS - Reg
highest, temporary types lowest — remains consistent across most years,
suggesting the pay hierarchy is structural rather than a recent
development. Any visible divergence in recent hire years would warrant
further investigation into whether new hires under temporary appointments
are falling further behind their regular counterparts over time.

year_appt = (
    df_filtered.groupby(['YEAR', 'APPT_TYPE'])['COMPRATE']
    .mean()
    .reset_index()
)

year_counts = df_filtered.groupby('YEAR').size()
valid_years = year_counts[year_counts >= 10].index
year_appt = year_appt[year_appt['YEAR'].isin(valid_years)]

fig, ax = plt.subplots(figsize=(13, 6))

for appt in year_appt['APPT_TYPE'].unique():
    subset = year_appt[year_appt['APPT_TYPE'] == appt]
    ax.plot(subset['YEAR'], subset['COMPRATE'],
            marker='o', markersize=4, label=appt, linewidth=2)

ax.set_title('Average Salary by Hire Year and Appointment Type',
fontsize=14)
ax.set_xlabel('Hire Year')
ax.set_ylabel('Average Annual Salary ($)')
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, _:
f'${x:,.0f}'))
ax.legend(title='Appointment Type')
ax.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```

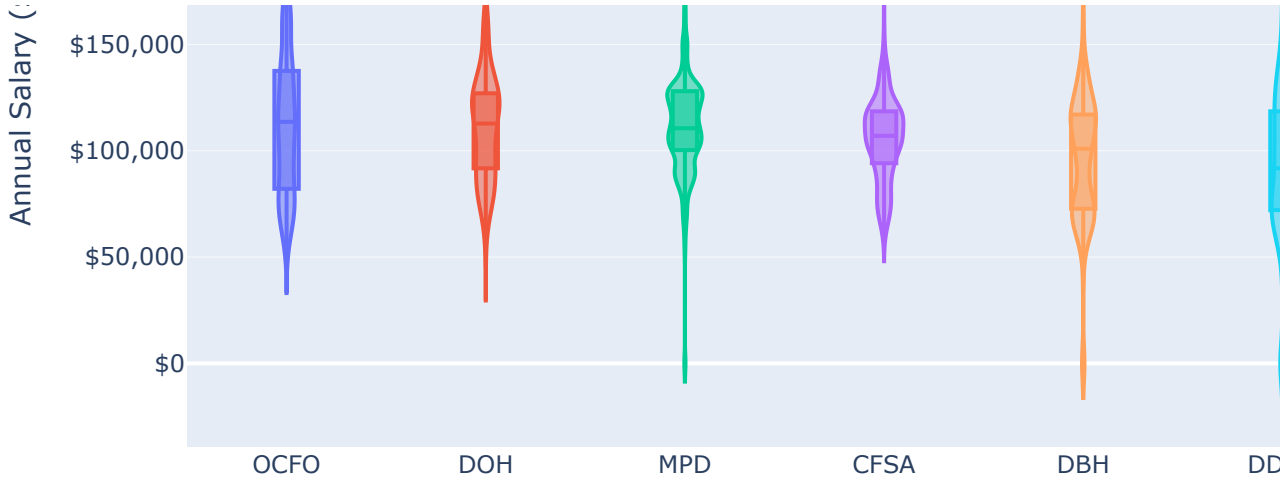
Out[47]:



```
# Intent: Bar charts of average salary hide distributional shape. This
interactive violin plot reveals the full spread of salaries within each
of the top 12 departments by headcount, making internal pay variation
visible alongside the between-department differences already identified.
# Interpretation: Departments with wide violins — those employing both
front-line workers and senior managers — show high internal pay variation
that averages would obscure. Departments with narrow, tall violins have
more homogeneous pay structures. The embedded box plot inside each violin
pinpoints the median and interquartile range, allowing readers to compare
both central tendency and spread simultaneously. This confirms that
internal equity — not just inter-agency equity — is a meaningful
dimension of the DC pay disparity problem.
top12_depts = df_filtered['DESCRSHORT'].value_counts().head(12).index
df_violin =
df_filtered[df_filtered['DESCRSHORT'].isin(top12_depts)].copy()

dept_order = (
    df_violin.groupby('DESCRSHORT')['COMPRATE']
    .median()
    .sort_values(ascending=False)
    .index.tolist()
)

fig = px.violin(
    df_violin,
    x='DESCRSHORT',
    y='COMPRATE',
    color='DESCRSHORT',
    box=True,
    points=False,
    category_orders={'DESCRSHORT': dept_order},
    title='Salary Distribution by Department (Top 12 by Headcount)',
    labels={
        'DESCRSHORT': 'Department',
        'COMPRATE': 'Annual Salary ($)'
    },
    hover_data=['APPT_TYPE', 'GRADE_NUM']
)
fig.update_layout(
    showlegend=False,
    yaxis_tickprefix='$',
    yaxis_tickformat=',.0f',
    xaxis_title='Department',
    yaxis_title='Annual Salary ($)'
)
fig.show()
```

Out[46]:





7. Insights and Conclusion

Research Questions call back

Research Questions Answered

Hire year vs. grade was partially answered: hire year is a weak predictor with a slightly positive trend, but grade composition changes over time are the more likely explanation. Appointment type and pay was answered clearly and affirmatively — multiple visualizations confirmed a systematic pay hierarchy by appointment type that operates independently of the grade structure.

Signiicant Patterns and Trends

Key findings appointment type outpredicts hire year; the distribution is right-skewed; department creates a third axis of disparity; the appointment type pay hierarchy has been consistent over decades; and internal departmental variation is high.

Inights from Visualization

Highlights the surprise deviation heatmap as the most analytically powerful chart, and explains what the KDE and violin plots revealed that simpler charts could not.

Limitatons

COMPRATE excludes total compensation; the data is cross-sectional not longitudinal; grade comparability across agencies is assumed; filtering excluded some

appointment types; and no demographic variables are available.

Future Steps

Linking to demographic data, longitudinal salary tracking, regression modeling, private sector benchmarking, and investigating temporary-to-regular appointment conversion rates.